

Development Plan

Software Engineering

Team #21, LingoLyrics

Student 1 name

Aansh Kotian

Student 3 name

Student 4 name

Table 1: Revision History

Date	Developer(s)	Change
Date1	Name(s)	Description of changes
Date2	Name(s)	Description of changes
...	...	...

This document outlines the Development Plan for Software Engineering (**LingoLyrics**), a multi-platform language-learning ecosystem that bridges interactive song playback with structured language instruction. This report covers project governance, workflow and repository strategies, continuous integration pipelines, schedule breakdowns, risk management for our Proof of Concept (PoC), technology stack selections, and team operational charters.

1 Confidential Information?

There is no confidential information associated with this project. LingoLyrics is an open-source capstone project operating without corporate non-disclosure agreements, proprietary industry datasets, or commercial restrictions.

2 IP to Protect

There is no proprietary intellectual property (IP) to protect. All application code, file format specifications (song map schema), system documentation, and design assets will be developed openly under standard university capstone guidelines.

3 Copyright License

The software source code, map schema definitions, and associated documentation will be distributed under the **MIT License**. This choice ensures maximum accessibility for community contributors and map creators while permitting open integration with third-party tools.

4 Team Meeting Plan

Meetings will be conducted using a hybrid approach, leveraging specific virtual channels and in-person sessions during scheduled capstone periods.

- **Internal Team Meetings:** Held virtually every Friday from 11:00 AM to 12:00 PM via Discord or Google Meet to review project progress, manage sprint planning, assign issues, and resolve technical blockers.
- **Supervisor Meetings:** Held periodically on Fridays from 10:00 AM to 10:30 AM via Microsoft Teams with our supervising professor, Dr. Angela Zavaleta Bernuy, to present milestone deliverables, track progress, and receive guidance.
- **Meeting Structure:** Meeting operational roles will rotate among team members each session:

- **Meeting Chair:** Prepares and distributes the agenda prior to the call.
- **Notetaker:** Records meeting minutes, key technical decisions, and actionable items directly into GitHub Discussions/Issues.

5 Team Communication Plan

Our team will use multiple channels to stay organized and collaborate effectively:

- **Discord** — for quick day-to-day communication, quick queries, and ad-hoc discussions.
- **GitHub Discussions & Issues** — for tracking technical tasks, bug reports, pull request reviews, and project codebase progress.
- **Notion** — for organizing project documents and resources, taking meeting notes, and task planning.

These tools will ensure that all updates, discussions, and decisions are documented and accessible to all team members.

6 Team Member Roles

All team members will be responsible for creating issues, developing, testing, documenting, and reviewing code.

Development work will be partitioned based on project features/components rather than individual technical expertise. For instance, when implementing the end product, one individual will be responsible for the end-to-end development of the song map editor or interactive lesson player components, including frontend UI, backend parsing, and schema integration. This allows technical expertise to be evenly distributed across the team, providing an equal learning and development opportunity for each teammate.

Administrative tasks will be broken down into the following operational roles, which will be cycled on a regular basis:

6.1 Project Lead

- Chairs meetings with the team and supervisors by preparing agendas and guiding discussions.
- Serves as the primary liaison with the supervisor, Dr. Angela Zavaleta Bernuy.
- Ensures the team is making steady progress toward weekly project goals and capstone milestones.

6.2 Content Manager

- Collects and manages materials needed for external approvals, custom document submissions, and ethics/copyright disclosures.
- Documents external datasets, audio libraries, language processing packages, and academic references utilized throughout the project.

6.3 Minute-Taker

- Takes detailed meeting minutes during internal team meetings and supervisor syncs.
- Highlights upcoming project deliverables, internal task deadlines, and submission requirements.
- Coordinates final document compilation and submits course deliverables.

6.4 DevOps & Resource Manager

- Manages repository configurations, branch protection policies, CI/CD automated pipelines, and cloud web-repository hosting resources.
- Oversees cross-platform compilation environments for desktop (Windows/macOS) and mobile (iOS/Android) targets.

7 Workflow Plan

To maintain high code health and release reliability, our engineering pipeline integrates automated continuous testing via GitHub Actions. Every pull request triggers CI jobs that execute unit test suites, run static analysis using **ESLint** (enforcing Airbnb TypeScript standards), and validate song-map JSON schema integrity. Branch protection rules strictly mandate that all status checks pass and at least one peer approval is logged before changes can be integrated.

Software development and project maintenance will be coordinated using GitHub for version control and issue management. All team activities, ranging from feature implementation to supervisor action items, are tracked through centralized issues. The scope of tracked work encompasses:

- Core Feature & Interface Engineering
- Automated Test Suite Construction
- Deliverable Compilation & Submission
- CI/CD Pipeline & Cross-Platform Target Configuration
- Defect Remediation & Bug Fixes

- Advisor Feedback Implementation
- Regular Synchronizations & Architecture Reviews

Work items involving code or document modifications must be tagged with a primary taxonomy label upon creation. Git feature branches are spawned directly from `develop` and must utilize the designated label as a name prefix, as structured in Table 2.

Table 2: Task Taxonomy and Git Branch Conventions

Taxonomy Label	Git Branch Pattern	Scope & Application
Docs	<code>docs/*</code>	Revisions to setup guides, system architecture specifications, and capstone reports.
Feat	<code>feat/*</code>	Constructing new capabilities within the lesson player, editor, or Anki export module.
Bug	<code>bug/*</code>	Fixing playback timing drift, parsing errors, or runtime exceptions.
Enhancement	<code>enhance/*</code>	Refactoring source code, optimizing audio performance, or refining UI components.
Infra	<code>infra/*</code>	Pipeline workflow updates, build target configurations, or dependency maintenance.

Feature branches must adhere to the naming convention `prefix/issue-id-short-description` (e.g., `feat/12-sync-player` or `bug/04-anki-payload`).

Sprint backlogs are refined during weekly planning syncs. Tasks are initialized using structured GitHub Issue templates requiring a comprehensive technical summary, acceptance criteria defining the Definition of Done (DoD), explicit owner and reviewer assignments, and linked dependencies. Developers work within isolated feature branches and write descriptive commit logs. Upon fulfilling the DoD and passing local lint/test checks, the author opens a Pull Request referencing the issue ID, addresses feedback from the assigned peer reviewer, and merges the work into `develop` once sign-off is complete.

8 Project Decomposition and Scheduling

Task progression and milestone tracking will be managed using GitHub Projects via a dedicated Kanban board to visualize development status. Swimlanes are

defined to capture each phase of the task lifecycle:

- **Backlog:** items that have been identified and logged for future iterations
- **To Do:** prioritized tasks scoped for the upcoming sprint, refined by team members
- **In Progress:** active development or documentation tasks underway
- **In Review:** implementation complete, awaiting peer feedback and automated test pass
- **Done:** Definition of Done (DoD) fully satisfied and merged into develop

Major deliverables and target deadlines for the term are outlined below:

- [Problem Statement](#), [PoC Plan](#), and [Development Plan](#) – September 28, 2026
- [SRS Document](#) and [Hazard Analysis](#) – October 19, 2026
- [V&V Plan](#) – November 2, 2026
- Design Document Revision -1 & Custom Deliverable Choice – November 16, 2026:
 - [System Architecture \(Module Guide\)](#)
 - [Detailed Design \(Module Interface Spec\)](#)
- [Team Contribution Survey and Productivity Report \(PoC\)](#) – November 20, 2026
- [Proof of Concept Demonstration](#) – Between November 23 and December 4, 2026
- Design Document Revision 0 – January 18, 2027:
 - [System Architecture \(Module Guide\)](#)
 - [Detailed Design \(Module Interface Spec\)](#)
- [Team Contribution Survey and Productivity Report \(Rev 0\)](#) – January 29, 2027
- [Revision 0 Demonstration](#) – Between February 1 and February 12, 2027
- [V&V Report and Custom Deliverable \(Winter\)](#) – March 8, 2027
- [Final Demonstration Revision 1](#) – Between March 15 and March 22, 2027
- [Team Contribution Survey and Productivity Report \(Rev 1\)](#) – March 19, 2027

- Final Documentation Package – March 31, 2027
- Video Submission & EXPO Poster – TBD / Late March 2027
- [Capstone EXPO Demonstration](#) – TBD

Link to GitHub project: <https://github.com/rhea333/LingoLyrics/tree/main>

9 Proof of Concept Demonstration Plan

The purpose of the Proof of Concept (PoC) plan is to outline how our team will validate the feasibility of LingoLyrics by testing and resolving its most critical technical risks prior to full system development.

9.1 Objective

The objective of our PoC is to demonstrate the technical feasibility of real-time audio-lyric synchronization, multi-language text segmentation, and direct flashcard export. The PoC does not need to deliver a fully styled end-to-end consumer application; rather, it should validate that the underlying timing, NLP tokenization, and export workflows operate reliably enough to justify full system implementation.

9.2 Scope of PoC

- **Core Deliverables:** A functional prototype that:
 - Plays back an audio track while maintaining sub-second, real-time highlighting of corresponding lyric lines across desktop and mobile browsers.
 - Accurately parses non-Latin or unspaced text (e.g., Japanese Kanji/Kana or Vietnamese) into interactive words with corresponding transliteration/romanization.
 - Generates and transmits a serialized vocabulary package (including target word, definition, and context sentence) directly into a local or remote Anki instance.
- **Exploratory Nature:**
 1. Evaluating cross-platform web audio timing APIs to achieve reliable sub-second synchronization latency.
 2. Comparing language-specific tokenization pipelines for accurate segmentation of non-spaced scripts.
 3. Testing seamless integration methods with Anki (e.g., AnkiConnect API versus custom file payloads).
- **Not Included:** Complete user account management, custom UI design/styling, broad catalog expansion, or cloud sync backends.

10 Expected Technology

We expect to use **TypeScript** as our primary programming language across the client application and backend service integration, ensuring type safety and code reusability. Python or Go may be utilized for backend microservices or specialized text processing tasks.

For application development, we plan to use **React Native** with **Expo** (or **Electron/React** for desktop targets) to build a responsive, cross-platform user interface. For language processing, we will integrate specialized text segmentation tools such as **kuromoji.js** or **Sudachi** to handle word tokenization, furigana generation, and romanization of non-Latin scripts.

Audio timing and synchronized lyric rendering will be handled using platform-agnostic playback engines such as **Howler.js** or the native Expo Audio API, capable of sub-second timekeeping precision.

We will use **ESLint** and **Prettier** to enforce consistent code formatting and coding standards, managed locally via **Husky** pre-commit hooks.

Testing will be conducted using **Jest** for unit and integration tests, alongside **istanbul/nyc** or built-in coverage tools to measure test completeness.

CI/CD & Version Control: We will use **Git**, **GitHub**, and **GitHub Projects** for version control, issue tracking, and sprint management. Continuous Integration (CI) pipelines will be executed via **GitHub Actions** to run automated test suites, lint checks, and cross-platform build verifications on every pull request.

11 Use of AI and LLMs

- **Code Development:** AI tools (e.g., GitHub Copilot, ChatGPT) will be utilized as programming assistants for generating boilerplate code, routine utility functions, and regular expressions for text parsing.
- **Documentation & Quality Assurance:** LLMs will be used exclusively for proofreading documentation—identifying grammatical errors, resolving text parsing inconsistencies, and ensuring proper LaTeX syntax formatting—as well as suggesting candidate unit test scenarios.
- **Verification Protocol:** No AI-generated code or text modifications will be integrated directly without human verification. All outputs must undergo thorough manual peer review, adhere to project linting standards, and pass automated unit test suites prior to being merged.

12 Coding Standard

All codebase components will strictly follow the ****Airbnb JavaScript/TypeScript Style Guide****. Adherence will be automatically enforced via **ESLint** pre-commit hooks and status checks in GitHub Actions.

13 Appendix — Generative AI Use Disclosure

13.1 AI Tools Used

- **Tool Name:** OpenAI ChatGPT / Anthropic Claude / Google Gemini.
- **Version:** Current Web Models.

13.2 How and Where Used

- **Proofreading and Quality Control:** Generative AI was **not** used to generate core documentation text or technical content directly. Instead, tools were strictly limited to proofreading existing team-written drafts, catching grammatical and syntactical errors, and refining text phrasing for clarity and consistency.

13.3 Verification of AI-Generated Content

All proofreading edits and technical terminology refinements provided by AI tools were systematically reviewed, edited, and verified by team members to guarantee complete accuracy, originality, and alignment with LingoLyrics project goals and course submission standards.

Appendix — Reflection

1. Why is it important to create a development plan prior to starting the project?

Creating a development plan prior to starting the project encourages the team to establish collaboration practices, align expectations, and identify technical risks early before writing code. It sets clear guidelines for communication, Git workflows, coding standards, and project scheduling, which significantly reduces misunderstandings and integration friction later. A comprehensive plan also provides a shared reference for team roles, decision-making protocols, and toolchains. By agreeing on these foundational practices upfront, the team saves time, prevents scope creep, and can focus directly on executing technical milestones such as real-time audio synchronization and multi-language tokenization.

2. In your opinion, what are the advantages and disadvantages of using CI/CD?

Advantages: CI/CD enforces coding standards automatically from day one. Automated build and test workflows instantly catch syntax errors, broken dependencies, and failing component logic before pull requests reach the main branch. It maintains a stable codebase, streamlines peer reviews through automated linting checks, and provides fast feedback on overall project health across diverse development environments.

Disadvantages: Pipeline configuration and maintenance require non-trivial setup time and technical overhead. Continuous integration scripts can slow down rapid prototyping if builds become slow or flaky. Furthermore, CI/CD is only as reliable as the underlying test suite; maintaining comprehensive test coverage demands ongoing team effort, and build runners can introduce minor resource bottlenecks during tight delivery windows.

3. What disagreements did your group have in this deliverable, if any, and how did you resolve them?

While organizing the Proof of Concept (PoC) plan and overall development strategy, our team had two key technical debates:

- **Target Platform (Web Application vs. Native Mobile App):** We debated whether to focus initial development on a web-based app or a native mobile application. The core issue was balancing low-latency, sub-second audio sync across platforms with team implementation bandwidth. We resolved this by agreeing on a web-first architecture that prioritizes cross-browser media timing APIs, while leaving mobile-responsive layout options open for expansion.
- **Primary Target Audience (Focused/Serious vs. Casual Learners):** We had differing views on whether the app should cater to

”sit-down” power users (who require deep grammar breakdowns, detailed segment tokenization, and direct Anki flashcard export) versus casual, on-the-go language learners. We resolved this by prioritizing serious self-directed learners as our primary persona for the PoC scope, while keeping the user interface clean and accessible enough for casual listening.

Appendix — Team Charter

External Goals

The team's main goal is to create an engaging, accessible tool that bridges music consumption and language acquisition. Through LingoLyrics, we aim to strengthen our technical capabilities in cross-platform web audio engineering, multi-language NLP tokenization, and third-party workflow integrations like Anki. We also hope to deliver a polished, technically rigorous project that stands out at the Capstone EXPO, serves as a strong portfolio piece for full-stack software development, and earns recognition from judges and peers.

Attendance

Expectations

The team expects all members to attend scheduled syncs and arrive on time. If a member is unable to attend or will be late, they are expected to notify the team in Discord ahead of time, review the recorded meeting notes, and provide an async status update on their assigned tasks. Prior to each meeting, the meeting chair will share an agenda outlining key discussion points (e.g., audio engine performance, map editor features, SRS integration). Meetings will end early if all agenda topics are resolved, or continue asynchronously in Discord if further technical discussion is needed.

Acceptable Excuse

An acceptable excuse for missing a meeting is an unpredictable personal emergency, illness, or documented academic exam conflict communicated ahead of time. Missing meetings due to poor time management or avoidable scheduling conflicts is unacceptable.

For project deliverables, acceptable excuses include unforeseen personal emergencies or severe technical blockers that were proactively flagged to the group. Unacceptable excuses include underestimating task complexity, poor time planning, or neglecting assigned components until right before deadline dates. We value transparency and expect everyone to communicate potential delays early so the team can adjust task allocations.

In Case of Emergency

Team members must notify the group via Discord as soon as practically possible during an emergency. For occasional occurrences, team members will redistribute critical tasks (such as urgent PR reviews or deliverable sections) to keep project development moving forward. However, if a member experiences repeated issues meeting deadlines, the team will schedule a check-in to re-balance responsibilities or consult course instructors if necessary.

Accountability and Teamwork

Quality

Deliverables and codebase additions must meet Level 4 standard criteria on project rubrics. Code merged into main branches must pass automated CI linting checks, unit tests, and receive approval from a designated peer reviewer. If team members encounter unfamiliar domain concepts—such as complex foreign text tokenizers or cross-platform web audio APIs—they are expected to research solutions independently and consult team members to maintain high technical standards.

Attitude

Team members must maintain a professional, supportive, and open-minded attitude. Feedback during pull request reviews or planning sessions should remain constructive and focused on code quality and project objectives rather than personal preferences.

Stay on Track

Each team member is accountable for completing assigned GitHub issues within sprint timeframes. Work progress will be tracked transparently using GitHub Projects and commit histories. If someone anticipates falling behind on a task, they must flag it early in Discord so workload can be shared. Individual contributions will be regularly reviewed through issue completion and active participation in code reviews to ensure equitable workloads.

Team Building

Our team will hold regular in-person meetings each week to foster strong working relationships. We also plan to dedicate brief informal time before sprint planning sessions and organize monthly team social activities to maintain high team morale and open communication.

Decision Making

Decisions will be made through technical consensus following research and discussion of trade-offs. If the team reaches an impasse on architectural choices (e.g., platform targets or language tokenization libraries), members will present brief technical comparisons, consult with project advisors if needed, and conduct a formal team vote to reach a binding decision.